



GRADO

PROGRAMACIÓN Y ESTRUCTURAS DE DATOS AVANZADAS

ENUNCIADO PRÁCTICA 2

Curso 2026-2027

GRADO EN INGENIERÍA EN INFORMÁTICA Y EN TECNOLOGÍAS DE LA INFORMACIÓN

1.- ENUNCIADO DE LA PRÁCTICA

Dadas dos cadenas de caracteres *X* e *Y*, definidas sobre un alfabeto finito, se desea determinar las modificaciones de coste total mínimo necesarias para transformar $X = x_1, x_2, \ldots, x_n$ con longitud *n* en $Y = y_1, y_2, \ldots, y_m$ con longitud *m*.

Para realizar la transformación se permiten las siguientes operaciones:

Borrado: eliminar un carácter de *X, con coste* c_b.
Inserción: añadir un carácter a *Y, con coste* c_i.
Sustitución: reemplazar un carácter de *X* por otro, *con coste* c_s.
Intercambio*: c*ambia la posición de un carácter de *X* por la posición de otro en *X*, con coste c_c.

La distancia de edición entre dos cadenas se define como el menor coste para convertir una cadena en la otra. Por ejemplo, para transformar **sacos** en **caso** son necesarias como mínimo dos operaciones:

1. **sacos** → **casos** — intercambio de los caracteres s y c.
2. **casos** → **caso** — borrado de s.

Por tanto, su distancia de edición es vendrá determinada por la suma de los costes $c_c + c_b$. Ten en cuenta que el número de pasos no determina la distancia de edición, sino la suma de los costes.

Se pide diseñar un algoritmo basado en Ramificación y Poda que resuelva el problema descrito entre dos cadenas cualesquiera. Para ello, se realiza una búsqueda de soluciones disponiendo de un criterio de poda de las ramificaciones que conducen a soluciones peores que las mejores conocidas. Para calcular las posibles extensiones válidas de un nodo hay que considerar las posibles situaciones de coincidencia entre la cadena *X* y la cadena *Y.*

2.- REALIZACIÓN DE LA PRÁCTICA

2.1.- Diseño del algoritmo

La práctica constará de un programa en java que resuelva el problema aplicando el esquema indicado, junto con una memoria de su implementación.

2.2.- Argumentos y parámetros

La práctica se invoca usando la siguiente sintaxis:

```
java  CosteCadenas [-t][-h] [fichero entrada] [fichero salida]
```

o

```
java –jar CosteCadenas.jar [-t][-h] [fichero entrada] [fichero salida]
```

Los argumentos son los siguientes:

- **-t**: traza cada paso de manera que se describa la aplicación del algoritmo utilizado.
- **-h**: muestra una ayuda y la sintaxis del comando.
- **fichero_entrada:** es el nombre del fichero del que se leen los datos.
- **fichero_salida:** es el nombre del fichero que se creará para almacenar la salida. Si el fichero ya existe, el comando dará un error. Si falta este argumento, el programa muestra el resultado por pantalla.

Por ejemplo:

```
$ java CosteCadenas -h  <ENTER>

SINTAXIS: CosteCadenas [-t][-h] [fichero entrada] [fichero salida]
     -t              Traza cada paso
     -h              Muestra esta ayuda
     [fichero entrada] Palabras X,Y y los costes
     [fichero salida] Pasos realizados, y coste final
```

2.3- Datos de entrada

Un fichero con los siguientes elementos en el mismo orden, empezando en la primera fila del mismo:

Palabra *X*
Palabra *Y*
Coste de borrado c_b
Coste de inserción c_i
Coste de sustitución c_s
Coste de intercambio c_c

2.4- Datos de salida

Se debe volcar a un fichero de texto los diferentes pasos realizados, y en una línea final, el coste obtenido. El siguiente texto se corresponde a un ejemplo de salida, dónde no hace falta indicar los datos entre paréntesis (solo se muestran con carácter informativo a continuación):

sacos (Palabra X*)*
casos
caso (Palabra Y)
2 (suponiendo c_b=c_c=*1)*

2.5.- Implementación del algoritmo

El programa se desarrollará en Java siguiendo un diseño orientado a objetos. Los detalles del entorno recomendado se encuentran en la guía de la asignatura. Se valorará el diseño OO y la eficiencia del desarrollo.

3.- CUESTIONES TEÓRICAS DE LA PRÁCTICA

1) Justifica la idoneidad del algoritmo seleccionado.
2) Explica las condiciones de poda utilizadas en el algoritmo y describe cómo se realiza la estimación del coste de las soluciones parciales.
3) Analiza el coste computacional del algoritmo.
4) Describe alternativas al esquema utilizado, si las hay, y compara su coste con el de la solución realizada.

4.- ENTREGA DE LA PRÁCTICA

4.1 Material que hay que entregar al Tutor

Se confeccionará una memoria, en PDF con el siguiente índice:

1. **Portada** de la memoria con **nombre, apellidos, dni** y **dirección de correo**.
2. Respuesta a las cuestiones teóricas planteadas en este enunciado.
3. Un ejemplo de ejecución para distintas posiciones del caballo.
4. Un listado del código fuente completo.

4.2 Normativa de las prácticas en relación con el Centro Asociado:

1. La asistencia a las sesiones de prácticas es obligatoria.
2. El calendario y procedimiento para asistir a las sesiones de prácticas **está publicado en su Centro Asociado o bien aparece en el foro correspondiente a su centro en el curso virtual.**
3. El plazo de entrega de la documentación y de la práctica **lo establece el Tutor de prácticas de cada Centro Asociado o Campus**.
4. El Tutor califica la práctica, informa al alumno y en su caso la revisa de acuerdo con los horarios y procedimiento que establezca el Centro Asociado.
5. Todos los alumnos deberán registrarse a través del Curso Virtual en el grupo del Tutor/a con el que hayan asistido a las sesiones presenciales obligatorias a fin de que su práctica pueda ser calificada.
6. La práctica se debe aprobar en la misma o anterior convocatoria para que se pueda calificar la asignatura. En caso contrario la calificación será de suspenso.
7. La práctica se entregará tanto en el entorno virtual como al Tutor. La falta de cualquiera de ellas será motivo suficiente para quedar excluida de la convocatoria.

El alumno debe asegurarse de que no se da ninguna de las siguientes circunstancias, ya que implican automáticamente una calificación de suspenso:

- **Código:** el código no compila, no está desarrollado en Java, no se corresponde con el pseudocódigo recogido en la documentación, no es original, está copiado de la red, academia, compañero, modelos de lenguaje o inteligencia artificial, etc., o no sigue un diseño OO encapsulado o modular.
- **Ejecutable:** el ejecutable no termina, se queda sin memoria con ejemplares pequeños o aborta sin justificación. El ejecutable no lee los ficheros previstos en el formato adecuado. No trata los argumentos o no se ajusta a las especificaciones.
- **Documentación:** No se presenta en el soporte indicado por el tutor o está incompleta.
- **Soporte:** No se puede leer, o contiene un virus de cualquier tipo. A este respecto, las prácticas en las que se detecte cualquier tipo de virus estarán suspensas.

Los **alumnos estudiando en el EXTRANJERO** se deberán poner en contacto con el profesor tutor que se indicará en los foros.